

SHETH LUJ & M V COLLEGE

PRACTICAL NO 2

AIM :

Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

CLI IMPLEMENTATION :

CODE :

```
from math import gcd
```

```
def mod_inverse(e, phi):  
    for d in range(2, phi):  
        if (d * e) % phi == 1:  
            return d  
    return None
```

```
p = int(input("Enter prime number p: "))  
q = int(input("Enter prime number q: "))
```

```
n = p * q  
phi = (p - 1) * (q - 1)
```

```
e = int(input("Enter public exponent e: "))
```

```
while gcd(e, phi) != 1:  
    print("e must be coprime with phi.")  
    e = int(input("Enter another value of e: "))
```

```
d = mod_inverse(e, phi)
```

```
print("\nPublic Key:", (e, n))  
print("Private Key:", (d, n))
```

```
message = int(input("\nEnter numeric message: "))
```

```
cipher = pow(message, e, n)
```

SHETH LUJ & M V COLLEGE

```
print("Encrypted Message:", cipher)
```

```
plain = pow(cipher, d, n)
```

```
print("Decrypted Message:", plain)
```

OUTPUT :

```
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun
MD64)] on win32
Enter "help" below or click "Help" above
.>>
===== RESTART: C:/Users/IT-
Enter prime number p: 17
Enter prime number q: 11
Enter public exponent e: 7

Public Key: (7, 187)
Private Key: (23, 187)

Enter numeric message: 88
Encrypted Message: 11
Decrypted Message: 88
.>> |
```

SHETH LUJ & M V COLLEGE

GUI ENCRYPTION AND DECRYPTION :

CODE :

```
import tkinter as tk
from tkinter import messagebox
from math import gcd

# ----- RSA FUNCTIONS ----- #
def mod_inverse(e, phi):
    for d in range(2, phi):
        if (e * d) % phi == 1:
            return d
    return None

def generate_keys():
    global e, d, n

    try:
        p = int(entry_p.get())
        q = int(entry_q.get())

        if p == q:
            messagebox.showerror("Error", "p and q must be different prime numbers.")
            return

        n = p * q
        phi = (p - 1) * (q - 1)

        e = 7
        while gcd(e, phi) != 1:
            e += 2

        d = mod_inverse(e, phi)
```

SHETH LUJ & M V COLLEGE

```
public_key.config(text=f"Public Key : ({e}, {n})")
private_key.config(text=f"Private Key : ({d}, {n})")

except:
    messagebox.showerror("Error", "Please enter valid prime numbers.")

def encrypt():
    try:
        message = int(entry_message.get())

        if message >= n:
            messagebox.showerror("Error", "Message must be smaller than n.")
            return

        cipher = pow(message, e, n)

        entry_cipher.delete(0, tk.END)
        entry_cipher.insert(0, str(cipher))

    except:
        messagebox.showerror("Error", "Generate keys first.")

def decrypt():
    try:
        cipher = int(entry_cipher.get())

        plain = pow(cipher, d, n)

        entry_plain.delete(0, tk.END)
        entry_plain.insert(0, str(plain))

    except:
        messagebox.showerror("Error", "Enter a valid cipher text.")
```

----- MAIN WINDOW -----

SHETH LUJ & M V COLLEGE

```
root = tk.Tk()
root.title("RSA Encryption & Decryption")
root.geometry("650x620")
root.configure(bg="#FCE4EC")
root.resizable(False, False)

# ----- HEADER ----- #
header = tk.Label(
    root,
    text="RSA Encryption & Decryption",
    bg="#E91E63",
    fg="white",
    font=("Arial", 20, "bold"),
    pady=12
)
header.pack(fill="x")

student = tk.Label(
    root,
    text="Developed By : Pushpal Pathak (T102)",
    bg="#FCE4EC",
    fg="#AD1457",
    font=("Arial", 12, "bold")
)
student.pack(pady=10)

# ----- INPUT FRAME ----- #
frame = tk.Frame(root, bg="#F8BBD0", bd=3, relief="ridge")
frame.pack(pady=15, padx=20)

tk.Label(frame, text="Prime Number (p)", bg="#F8BBD0",
        fg="#880E4F", font=("Arial", 11, "bold")).grid(row=0, column=0,
        padx=15, pady=10)

entry_p = tk.Entry(frame, font=("Arial", 11), width=22)
```

SHETH LUJ & M V COLLEGE

```
entry_p.grid(row=0, column=1)
```

```
tk.Label(frame, text="Prime Number (q)", bg="#F8BBD0",  
         fg="#880E4F", font=("Arial", 11, "bold")).grid(row=1, column=0,  
padx=15, pady=10)
```

```
entry_q = tk.Entry(frame, font=("Arial", 11), width=22)  
entry_q.grid(row=1, column=1)
```

```
generate_btn = tk.Button(  
    frame,  
    text="Generate Keys",  
    bg="#EC407A",  
    fg="white",  
    font=("Arial", 11, "bold"),  
    width=20,  
    command=generate_keys  
)  
generate_btn.grid(row=2, column=0, columnspan=2, pady=15)
```

```
# ----- KEY DISPLAY ----- #
```

```
public_key = tk.Label(  
    root,  
    text="Public Key :",  
    bg="#FCE4EC",  
    fg="#880E4F",  
    font=("Arial", 11, "bold")  
)  
public_key.pack()
```

```
private_key = tk.Label(  
    root,  
    text="Private Key :",  
    bg="#FCE4EC",  
    fg="#880E4F",
```

SHETH LUJ & M V COLLEGE

```
        font=("Arial", 11, "bold")
    )
    private_key.pack(pady=5)

    # ----- MESSAGE ----- #
    tk.Label(
        root,
        text="Plain Message (Number)",
        bg="#FCE4EC",
        fg="#880E4F",
        font=("Arial", 11, "bold")
    ).pack()

    entry_message = tk.Entry(root, width=30, font=("Arial", 11))
    entry_message.pack(pady=5)

    encrypt_btn = tk.Button(
        root,
        text="Encrypt",
        bg="#FF80AB",
        fg="white",
        font=("Arial", 11, "bold"),
        width=18,
        command=encrypt
    )
    encrypt_btn.pack(pady=10)

    # ----- CIPHER ----- #
    tk.Label(
        root,
        text="Cipher Text",
        bg="#FCE4EC",
        fg="#880E4F",
        font=("Arial", 11, "bold")
    ).pack()
```

SHETH LUJ & M V COLLEGE

```
entry_cipher = tk.Entry(root, width=30, font=("Arial", 11))
entry_cipher.pack(pady=5)
```

```
decrypt_btn = tk.Button(
    root,
    text="Decrypt",
    bg="#C2185B",
    fg="white",
    font=("Arial", 11, "bold"),
    width=18,
    command=decrypt
)
decrypt_btn.pack(pady=10)
```

```
# ----- DECRYPTED MESSAGE ----- #
tk.Label(
    root,
    text="Decrypted Message",
    bg="#FCE4EC",
    fg="#880E4F",
    font=("Arial", 11, "bold")
).pack()
```

```
entry_plain = tk.Entry(root, width=30, font=("Arial", 11))
entry_plain.pack(pady=5)
```

```
# ----- FOOTER ----- #
footer = tk.Label(
    root,
    text="Computer Science Practical • RSA Algorithm",
    bg="#E91E63",
    fg="white",
    font=("Arial", 10, "bold"),
    pady=8
)
```


SHETH LUJ & M V COLLEGE

```
)  
footer.pack(side="bottom", fill="x")  
  
root.mainloop()
```

OUTPUT :

ENCRYPTION :

Python 3.11.0 (tags/v3.11.0:cc50cc0, Oct 18 2020, 10:20:10) [AMD64] on win32

Ent MD6411 on win32

Ent RSA Encryption & Decryption

====

Ent

Ent

Ent

Ent

Pub

Pri

Ent

Enc

Dec

====

Ent

Ent

Ent

====

====

RSA Encryption & Decryption

Developed By : Pushpal Pathak (T102)

Prime Number (p)

Prime Number (q)

Generate Keys

Public Key : (7, 187)

Private Key : (23, 187)

Plain Message (Number)

Encrypt

Cipher Text

Decrypt

Decrypted Message

SHETH LUJ & M V COLLEGE